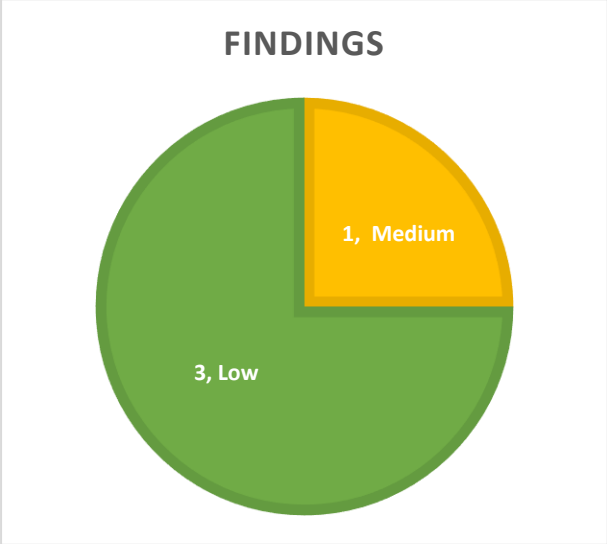
TrustSec

Smart Contract Audit

Morpho – Bundles

10/07/2026

Executive summary

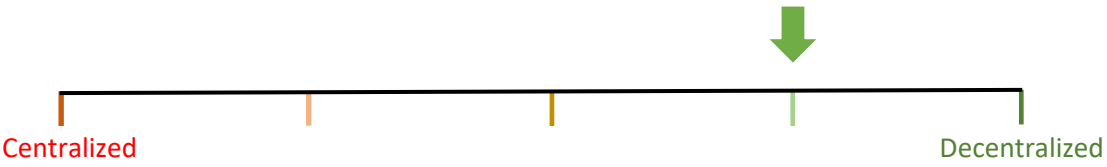


Category	Lending
Audited file count	1
Auditor	Trust HollaDieWaldfee
Time period	20/05/2026 - 05/06/2026

Findings

Severity	Total	Fixed	Acknowledged
High	0	0	0
Medium	1	0	1
Low	3	3	0

Centralization score



Signature

Digitally signed by Trust
Date: 2026.07.10 11:42:41+03'00'

Document properties

Versioning

Version	Date	Description
0.1	05/06/2026	Client report
0.2	10/07/2026	Mitigation review

Contact

Trust

trust@trust-security.xyz

Introduction

TrustSec has conducted an audit at the customer's request. The audit is focused on uncovering security issues and additional bugs contained in the code defined in scope. Some additional recommendations have also been given when appropriate.

Scope

- src/periphery/MidnightBundles.sol

Prior to the mitigation review, the bundles logic has been moved into a separate bundles repository:

- src/midnight/MidnightBundlesV1.sol
- src/libraries/TokenLib.sol

Repository details

- **Repository URL:** https://github.com/morpho-org/midnight
- **Commit hash:** 6783b978361f330de292715abf72f5f38bcb8523

Prior to the mitigation review, *MidnightBundles.sol* has been moved into a separate repository. The mitigations have been reviewed at the following commit:

- **Bundles repository URL:** https://github.com/morpho-org/bundles
- **Bundles mitigation hash:** 55bcda615e4b05acec958f16e58c681ca750a0bb

About TrustSec

TrustSec has been established by top-end blockchain security researcher Trust, in order to provide high quality auditing services. Trust is the leading auditor at competitive auditing service Code4rena, reported several critical issues to Immunefi bug bounty platform and is currently a Code4rena judge.

About the Auditors

Trust has established a dominating presence in the smart contract security ecosystem since 2022. He is a resident on the Immunefi, Sherlock and C4 leaderboards and is now focused in auditing and managing audit teams under TrustSec. When taking time off auditing & bug hunting, he enjoys assessing bounty contests in C4 as a Supreme Court judge.

HollaDieWaldfee is a distinguished security expert with a track record of multiple first places in competitive audits. He is a Lead Auditor at TrustSec and Senior Watson at Sherlock.

Disclaimer

Smart contracts are an experimental technology with many known and unknown risks. TrustSec assumes no responsibility for any misbehavior, bugs or exploits affecting the audited code or any part of the deployment phase.

Furthermore, it is known to all parties that changes to the audited code, including fixes of issues highlighted in this report, may introduce new issues and require further auditing.

Methodology

In general, the primary methodology used is manual auditing. The entire in-scope code has been deeply looked at and considered from different adversarial perspectives. Any additional dependencies on external code have also been reviewed.

Qualitative analysis

Metric	Rating	Comments
Code complexity	**Good**	*MidnightBundles* is a minimal and simple periphery contract.
Documentation	**Excellent**	Project is well documented with a whitepaper and code comments.
Best practices	**Excellent**	Project adheres to best practices.
Centralization risks	**Excellent**	Protocol is permissionless, only using privileged roles for fee settings and tick spacings. Fees have tight upper bounds.

Findings

Medium severity findings

TRST-M-1 *MidnightBundles* take attempts lack per-attempt gas limits

- **Category:** Griefing issues
- **Source:** MidnightBundles.sol
- **Status:** Acknowledged

Description

MidnightBundles is designed to opportunistically fill a route across multiple offers. Each bundle loop wraps individual *take()* attempts in **try/catch**, allowing the bundle to continue when one offer is no longer fillable or otherwise reverts:

```
try IMidnight(MIDNIGHT)
    .take(takes[i].offer, unitsToTake, taker, address(0), address(0), "",
takes[i].ratifierData) returns (
    uint256 resBuyerAssets, uint256
) {
    filledUnits += unitsToTake;
    filledBuyerAssets += resBuyerAssets;
} catch {}
```

However, these calls do not impose a per-attempt gas limit. A failed offer can therefore consume all gas forwarded to the core *take()* call before reverting (through a gasbombing attack on the buyer or seller callback), leaving only the 1/64th gas retained in MidnightBundles by EIP-150 for the caller. The **catch** block can still execute, but later valid offers in the route may no longer have enough gas to run.

For a single raw *take()* call, the transaction gas limit bounds the caller's exposure to one untrusted offer. However, in *MidnightBundles*, the lack of a per-offer gas budget means a single malicious offer can consume gas intended for many later fallback offers. This defeats the purpose of the **try/catch** route which is intended to allow individually reverting or misbehaving offers.

To illustrate the issue, suppose a legitimate later offer requires 200,000 gas to execute, and a malicious earlier offer consumes all gas it receives before reverting. Because EIP-150 leaves the caller with only 1/64 of the gas at the external call boundary, the caller would need more than 12,800,000 gas remaining before the malicious attempt to guarantee 200,000 gas remains afterward. With multiple malicious attempts or larger later routes, the wasted gas grows accordingly, quickly exceeding even block gas limits.

An attacker can systematically exploit the bundler and routing infrastructure by posting attractive offers that routers are likely to place early in the route. Although Midnight is agnostic to the quote matching layer, it should provide a sufficient foundation for untrusted, public matching.

Furthermore, even if there is a whitelisting of certain callback behaviors in the quote matching layer, the actual callback behavior at execution time can’t be determined in advance. Only

allowing immutable callback behaviors defeats the just-in-time purpose of callbacks as described in the whitepaper:

"Offers may specify a callback that executes at take time. This lets makers source the funds, or collateral the trade requires only when the offer is filled — instead of provisioning their side of the positions in advance.

In particular, makers can keep the capital backing their offers deployed productively elsewhere until their offer is filled. For example, a lender may keep assets deployed in a variable lending market (typically Morpho Blue) while quoting a fixed-rate offer on Midnight. If the offer is taken, the callback withdraws the necessary funds, provided sufficient liquidity is available, and completes settlement within the same transaction.

Callbacks are also particularly useful for rolling fixed-maturity exposure. A borrower approaching maturity may use a callback to buy back or repay debt in the current obligation and enter a latermaturity obligation atomically. Likewise, a lender may roll credit exposure from one maturity into another without first withdrawing into idle balances."

Recommended mitigation

Add per-attempt gas controls to the bundle functions. For example, each **Take** could include a **gasLimit** field, or the bundle functions could accept a global **maxGasPerTake** value. The bundler would then call **MIDNIGHT** with an explicit gas limit. By providing such gas limits in addition to the existing **try/catch**, the intention of *MidnightBundles* to decouple the success of individual offers from the overall bundle is realized.

Team response

Acknowledged. Callbacks are whitelisted on the offchain side and will be checked for gas attacks.

Low severity findings

TRST-L-1 Bundle buy functions do not bound continuous-fee slippage

- **Category:** Race conditions
- **Source:** MidnightBundles.sol
- **Status:** Fixed

Description

MidnightBundles exposes two buy-side bundle functions with slippage limits for immediate trade amounts:

```
function buyWithUnitsTargetAndWithdrawCollateral(
    uint256 targetUnits,
    uint256 maxBuyerAssets,
    ...
) external

function buyWithAssetsTargetAndWithdrawCollateral(
    uint256 targetBuyerAssets,
    uint256 minUnits,
    ...
) external
```

These checks bound the raw units and loan-token payment, but they do not bound the continuous fee attached to newly acquired credit. A buyer's economic output is closer to **credit - pendingFee** than to raw **credit** alone.

In *Midnight.take()*, the buyer's pending fee is computed from the market's current **continuousFee**:

```
uint128 buyerPendingFeeIncrease =
    UtilsLib.toUint128(buyerCreditIncrease.mulDivDown(_marketState.continuousFee *
timeToMaturity, WAD));
```

Midnight can provide buyer-side execution checks through the **takerCallback**, but both bundle buy functions hardcode that callback to **address(0)**:

```
IMidnight(MIDNIGHT).take(
    takes[i].offer,
    unitsToTake,
    taker,
    address(0),
    address(0),
    "",
    takes[i].ratifierData
)
```

As a result, a **continuousFee** increase before the bundle transaction is executed can leave slippage parameters satisfied while increasing **pendingFee** and reducing the buyer's net face value.

The impact is bounded by **MAX_CONTINUOUS_FEE** and the remaining time to maturity. However, *MidnightBundles* is the user-facing periphery that provides slippage protection, and users may reasonably expect those slippage controls to cover the economic value of the acquired credit rather than only the raw trade amounts.

Recommended mitigation

Add a buyer-side continuous-fee slippage bound to the bundle buy functions. One way to implement this is to have *MidnightBundles* pass itself as **takerCallback** for buy-side takes, accumulate the **pendingFeeIncrease** values received in *onBuy()*, and enforce the user-provided limit. This recommendation ties in with the previous finding TRST-L-4 which shows that the callbacks in their current implementation don't provide sufficient information and must be extended as well. Furthermore, accepting callbacks in *MidnightBundles* introduces new reentrancy attack surface that must be protected against.

Team response

Fixed in commit 0ce95c1.

Mitigation review

Verified, all buy and sell functions in *MidnightBundlesV1* now accept a **maxContinuousFee** parameter which is checked before each *take()*.

TRST-L-2 Sell bundles do not let takers bound resulting debt exposure

- **Category:** Race conditions
- **Source:** MidnightBundles.sol
- **Status:** Fixed

Description

The sell-side *MidnightBundles* functions let a taker sell credit in exchange for loan-token assets. A natural use case is for a lender to close a credit position by calling *supplyCollateralAndSellWithUnitsTarget()* with **targetUnits** equal to the lender's expected remaining credit.

However, a lender's effective credit can decrease before the bundle executes. This can happen because *take()* first updates the seller position, applying pending bad-debt slashing and continuous-fee accrual:

```
if (hasCredit(id, seller)) _updatePosition(offer.market, id, seller);
```

If the taker's credit has fallen below the **units** being sold, the excess **units - credit** become new debt. This is correct behavior in core *Midnight*, but it is unexpected and unintended for a periphery close-position flow where the taker intended to end with neutral exposure rather than silently cross into debt.

Raw *take()* can be wrapped with a taker callback. For maker buy offers, the taker is the seller and can pass a **takerCallback** that checks the resulting position during *onSell()* and reverts if the trade created debt. The units-target sell bundle does not expose this protection. It hardcodes the taker callback to **address(0)**:

```
IMidnight(MIDNIGHT).take(
    takes[i].offer,
    unitsToTake,
    taker,
    address(this),
    address(0),
    "",
    takes[i].ratifierData
)
```

After the loop, the bundle only checks that the requested **targetUnits** were filled and that the received assets satisfy **minSellerAssets**:

```
require(filledUnits == targetUnits, OutOfOffers());

uint256 referralFeeAssets = filledSellerAssets.mulDivDown(referralFeePct, WAD);
require(filledSellerAssets - referralFeeAssets >= minSellerAssets,
SellerAssetsTooLow());
```

The asset-target sell bundle has the same issue. It exposes **maxUnits**, and a user could set **maxUnits** based on their expected credit to avoid debt. However, if credit drops before execution, **filledUnits <= maxUnits** can still hold while some of those filled units exceed current credit and become debt. This path is less directly aligned with a close-position UX because the primary target is **targetSellerAssets**, but it has the same missing resulting-exposure bound.

Since executing timing is generally unpredictable, takers cannot use the periphery to neutralize their positions.

For the bundler's buy-side functions, the issue does not exist. Overshooting a user's debt is harmless since credit is an asset, not a liability.

Recommended mitigation

There are several possible approaches, with different tradeoffs.

The simplest protection is to add an explicit resulting-exposure guard to the sell bundle functions. For example, accept a **maxDebtAfter** parameter and revert unless the taker's debt after the bundle is at most that value:

```
require(IMidnight(MIDNIGHT).debtOf(id, taker) <= maxDebtAfter, DebtTooHigh());
```

This does not make the close exact, but it prevents the surprising outcome where the transaction succeeds while creating more debt than the user was willing to accept. It is also easy to reason about because it preserves the existing **targetUnits**, **targetSellerAssets**, **minSellerAssets**, and **maxUnits** semantics.

For the units-target close-position use case in *supplyCollateralAndSellWithUnitsTarget()*, the protocol could support an optional **shouldClose** mode. In that mode, the bundle would call *updatePosition()* before the take loop, cap **targetUnits** to the taker's fresh credit, and then enforce a final debt bound:

```
uint256 debtBefore;
if (shouldClose) {
    (uint128 freshCredit,,) = IMidnight(MIDNIGHT).updatePosition(market, taker);
    if (targetUnits > freshCredit) targetUnits = freshCredit;
    debtBefore = IMidnight(MIDNIGHT).debtOf(id, taker);
}

// take loop

if (shouldClose) {
    require(IMidnight(MIDNIGHT).debtOf(id, taker) <= debtBefore, DebtTooHigh());
}
```

The final debt check is important because a pre-loop clamp is not sufficient by itself. Credit cannot shrink further from fee accrual within the same transaction because **block.timestamp**

is fixed, but bad debt can still be realized during the bundle through callbacks on filled offers. A final debt guard protects against those in-transaction changes.

The main tradeoff of **shouldClose** is slippage semantics. If **targetUnits** is reduced, keeping the original **minSellerAssets** value is conservative and simple, but may cause the transaction to revert because fewer units are being sold. Adjusting **minSellerAssets** pro-rata can make the close mode more usable, but it changes the meaning of the user's slippage bound and can leak value when routes are ordered from best to worst. A bundle that takes fewer units should expect lower slippage than a bundle that takes more units.

The asset-target function is less naturally aligned with close-position semantics because the primary target is **targetSellerAssets** rather than selling all remaining credit.

Team response

Fixed in commit ecb1f3a.

Mitigation review

Verified, the mitigation adds a **reduceOnly** flag to all buy and sell functions in *MidnightBundlesV1*. If **reduceOnly=true**, sell functions restrict **unitsToTake** to the user's credit and buy functions restrict **unitsToTake** to the user's debt. If **unitsToTake** exceeds credit/debt the execution reverts. This means changes in the credit/debt in between submission of the transaction and execution can make bundle execution revert. One example is when realized bad debt lowers the lenders credit. *MidnightBundlesV1* does not support dynamic sizing at execution time.

Furthermore, note that the exposure after calling *take()* may still cross into debt for sells and credit for buys if the user has other open routes through which a take can be executed in a callback. For example, the user may have standing offers without debt/credit crossing restrictions.

TRST-L-3 Bundle functions lack deadlines to protect against delayed execution

- **Category:** Time sensitivity issues
- **Source:** MidnightBundles.sol
- **Status:** Fixed

Description

The four buy and sell functions in *MidnightBundles* execute multi-offer routes without a taker-controlled deadline:

```
function buyWithUnitsTargetAndWithdrawCollateral(
    uint256 targetUnits,
    uint256 maxBuyerAssets,
    ...
) external

function supplyCollateralAndSellWithUnitsTarget(
    uint256 targetUnits,
    uint256 minSellerAssets,
    ...
) external

function buyWithAssetsTargetAndWithdrawCollateral(
    uint256 targetBuyerAssets,
    uint256 minUnits,
    ...
) external

function supplyCollateralAndSellWithAssetsTarget(
    uint256 targetSellerAssets,
    uint256 maxUnits,
    ...
) external
```

The functions expose amount-based limits, but they do not let the taker specify a maximum execution time for the entire bundle.

This matters because a bundle transaction can remain pending longer than intended, including during gas spikes, downtime, or censorship attacks. When the transaction eventually executes, the route may still satisfy the amount-based checks while no longer matching the taker's intended market conditions. As maturity approaches, the fair market price is expected to increase. E.g., this can make sellers worse off even if they receive their **minSellerAssets**.

The offer-level **expiry** field does not address this issue because it is selected by the maker and applies to individual offers. It does not give the taker a direct way to say that the entire route should become invalid after a specific timestamp.

Recommended mitigation

It is recommended to add a **deadline** parameter to each user-facing bundle function and check it at the beginning of execution.

Team response

Fixed in commit 65516a6.

Mitigation review

Verified, the recommendation has been implemented.

Additional recommendations

TRST-R-1 Clarify full-repay referral-fee formula

MidnightBundles.repayAndWithdrawCollateral() documents the amount of loan-token assets that should be passed to fully repay a debt **D** when a referral fee is charged:

```
/// @dev To fully repay a debt D, pass assets = floor(D * WAD / (WAD - pct)).
```

The documented value is sufficient to fully repay **D**, but it is not always the minimum sufficient value. The function first rounds the referral fee down, then treats the remainder as the repaid units:

```
uint256 referralFeeAssets = assets.mulDivDown(referralFeePct, WAD);
uint256 units = assets - referralFeeAssets;
```

Because **referralFeeAssets** is rounded down, several different **assets** values can map to the same repaid **units** amount. Integrators that use the documented formula as an exact quote can therefore transfer more loan-token assets than necessary, with the excess paid to **referralFeeRecipient** as referral fee.

The discrepancy is not capped at one wei. At the maximum allowed **referralFeePct = WAD - 1**, repaying **D = 1** with the documented formula uses **1e18** assets, while **1** asset is already sufficient because the rounded-down referral fee is zero.

It is recommended to clarify that the documented formula is sufficient but not minimal. For example:

```
/// @dev Passing assets = floor(D * WAD / (WAD - pct)) is sufficient to fully repay a
debt D, but may
/// not be the minimum sufficient amount because referral fees round down.
```

TRST-R-2 Missing code-existence check in *_safeApprove()*

SafeTransferLib.safeTransfer() and *SafeTransferLib.safeTransferFrom()* both check that the token address contains code before calling it:

```
require(token.code.length > 0, NoCode());
```

However, *MidnightBundles._safeApprove()* calls the token directly without the same check:

```
function _safeApprove(address token, address spender, uint256 value) internal {
    (bool success, bytes memory returndata) =
token.call(abi.encodeCall(IERC20.approve, (spender, value)));
    if (!success) {
        assembly ("memory-safe") {
            revert(add(returndata, 0x20), mload(returndata))
        }
    }
    require(returndata.length == 0 || abi.decode(returndata, (bool)));
}
```

It is recommended to add the same code-existence check to *_safeApprove()* for consistency with the transfer helpers.

TRST-R-3 Update inaccurate or incomplete comments

The *MidnightBundles* comments for the multi-take functions say:

```
/// @dev This function should only be called with the same market for all takes.
```

The implementation requires **takes** to be non-empty because it reads **takes[0]**, and it checks that later offers use the same market. This is therefore a must, not only a recommended calling pattern. It is recommended to use:

```
/// @dev takes must be non-empty and all takes must use the same market, otherwise
the function reverts.
```

The *MidnightBundles._forceApproveMax()* comment says:

```
/// @dev Skips the approval entirely when the current allowance is already 2^95 - 1.
```

The code skips when the allowance is at least the threshold. It is recommended to update the comment:

```
/// @dev Skips the approval entirely when the current allowance is already at least
2^95 - 1.
```

TRST-R-4 Document that force-approval threshold can theoretically be exhausted by reentrant callbacks

MidnightBundles._forceApproveMax() replenishes an approval to **type(uint256).max** when the current allowance is below **type(uint96).max / 2**. Therefore, the functions should be compatible with token amounts less than or equal to this threshold:

```
function _forceApproveMax(address token, address spender) internal {
    if (IERC20(token).allowance(address(this), spender) >= type(uint96).max / 2)
return;
    _safeApprove(token, spender, 0);
    _safeApprove(token, spender, type(uint256).max);
}
```

This is a practical gas optimization and failsafe, so the bundler avoids resetting and re-approving on every call while still replenishing in the theoretical case that allowance has dropped below the threshold.

However, the way in which approval Is increased at the beginning of the function is an unsafe pattern. *Midnight.take()* can execute maker-controlled callbacks which reenter the bundler and consume the approval. Once the outer context continues, it encounters an exhausted approval and reverts. This theoretical attack vector should be documented. In practice it is not reachable because an approval of **type(uint256).max** can only be exhausted theoretically.

TRST-R-5 *safeApprove()* incorrectly describes token transfer as checking code size

It has been documented as a fix for TRST-R-2 that a code existence check in *safeApprove()* is unnecessary since a transfer always does it in the same call. This is incorrect. If the transfer happens via **PERMIT2** then there's no revert. The actual blocker is that *safeApprove()* is only used in *forceApproveMax()* which performs a high level *allowance()* call and reverts if there's no return value.

Centralization risks

TRST-CR-1 Authorized addresses are fully trusted by the authorizer

Users can grant permissions to act on their behalf through Midnight's **isAuthorized** mapping. An authorized address has full control over the authorizer's Midnight account and can call all *MidnightBundles* functions on behalf of the authorizer.

Systemic risks

TRST-SR-1 External token risks and assumptions

The protocol relies on strong assumptions about all loan tokens and collateral tokens. These are documented directly in the *Midnight* contract.

Since market creation is permissionless, any token can be used to create a market. The risk (e.g., depeg risk, liquidity risk, centralization risk) of external tokens must be assessed on a per-market basis.

The security boundary is the market. One bad token in a market can compromise the whole market but not spread to other markets that don't depend on the bad token.

TRST-SR-2 Oracle risks

Midnight relies on external oracles for collateral valuation. Incorrect, stale, or reverting oracle prices can affect market liveness and pricing. A detailed description of the oracle dependencies, and consequences of liveness and pricing issues, is provided in the *Midnight* contract.

Like for external tokens, the security boundary is the market. It is not possible for a misbehaving oracle in one market to affect another market that doesn't use the same oracle.

TRST-SR-3 Protocol allows for LLTV=WAD with zero collateral buffer

Markets using collateral with **LLTV = WAD** have a special risk profile. They allow borrowing with no overcollateralization buffer, so even small price movements, rounding effects, or liquidation delays can push a position into bad debt.

For such collaterals, the liquidation incentive factor is also **WAD**, meaning liquidators repay at the oracle price plus rounding effects and do not receive an economic liquidation premium. This disables the economic incentive to liquidate positions.

Because there is no collateral buffer, unhealthy positions in **LLTV = WAD** markets will often realize bad debt when liquidated. Lenders in such markets are therefore exposed to oracle error, price movement, execution delays, and liquidation liveness in a qualitatively different way compared to other configurations.